

TP1 - Gatolândia

17 pontos

Entrega: 05/04/2026

1 Objetivo do trabalho

Neste trabalho, vamos exercitar tópicos relativos ao primeiro terço do curso: grafos e alguns algoritmos elementares nessas estruturas.

Serão fornecidos alguns casos de teste para que você possa testar seu programa, mas eles não são exaustivos! Podem haver situações que não são ilustradas por eles; cabe a você pensar em novos casos e garantir que seu programa esteja correto e que ele implemente um algoritmo de complexidade adequada.

1.1 Informações importantes

O código fonte do seu trabalho deve estar contido em um **único** arquivo na linguagem C++ e deve ser submetido via Moodle na tarefa **Entrega TP1** até o dia 05/04/2026. Você terá ∞ tentativas para conseguir a nota total de execução; apenas a última submissão será levada em conta para fins de avaliação. Você não terá acesso a todos os casos de teste; determinar estratégias para testar seu programa e suas ideias faz parte do trabalho. Envios com atraso serão aceitos; leia a Seção 5 para a política de atrasos.

Plágio de qualquer natureza não será tolerado. Caso qualquer cola seja encontrada, seu *SEMESTRE* será zerado e as demais providências cabíveis serão tomadas. Escreva seu próprio código, de maneira legível e com comentários apropriados; ele pode ser útil no futuro próximo. **Não use nenhum LLM ou tecnologia similar para gerar código; as consequências serão as mesmas do plágio.**

2 Definição do problema

A Gatolândia reúne todos os membros da espécie *Felis catus* (os gatinhos domésticos) existentes na Terra, e é reconhecida internacionalmente como o único estado-nação com relações diplomáticas amigáveis com todos os países do mundo. Ela também é conhecida pela lealdade de seus habitantes; não estamos falando para com seus tutores, já que esses já são súditos naturais dos gatos, mas sim *entre* os gatinhos do mundo. Em particular, cada gato é diretamente leal a apenas um outro gato do mundo, mas um gato pode receber a lealdade de vários outros. A lealdade também é uma propriedade transitiva; isso foi conquistado com muito suor e esforço após o grande cisma felino de 1287, mas deixaremos essa história para o TP2. Um conjunto de gatos leais uns aos outros é chamada de *gataria* e hoje formam a base da sociedade felina. Cansada de ter de descobrir a qual gataria um dado indivíduo pertence (afinal, quanto mais trabalho menos sonecas), a gata Ariel decidiu romper com as tradições da Gatolândia, e quer forjar contratos de lealdade adicionais para que todos os gatinhos façam parte de uma única gataria, coexistindo em paz e preguiça.

Escreva um programa que ajude Ariel a fazer isso e voltar para seu cochilo da tarde no sofá! Ele deve receber a descrição de todos os vínculos de lealdade da Gatolândia e retornar um conjunto de tamanho mínimo de vínculos de lealdade novos que devem ser forjados por Ariel para satisfazer seus anseios preguiçosos.

3 Dicas

Um segundo equivale a, aproximadamente, 10^8 operações em um computador moderno. Ou seja, um algoritmo $\mathcal{O}(N)$ para $N \leq 10^6$ cabe em um segundo, mas um algoritmo $\mathcal{O}(N^2)$ não cabe.

Operações de entrada e saída são extremamente caras: use as funções `scanf` e `printf` ao invés das funções `cin`, `cout`.

4 Casos de teste

4.1 Formato da Entrada

Cada caso de teste é composto por duas linhas, que representam os vínculos de lealdade da Gatolândia. A primeira linha contém um único inteiro, N , que representa o número de gatos. É garantido que $2 \leq N \leq 10^6$. A próxima linha contém N inteiros; o i -ésimo desses inteiros, g_i , indica que o gato i é leal ao gato g_i ; é garantido que $g_i \neq i$ e $1 \leq g_i \leq N$.

4.2 Formato da Saída

Preste muita atenção a essa seção.

A primeira linha da saída deve conter um único inteiro F , que representa o número de vínculos de lealdade forjados por Ariel. Devem-se seguir F linhas. A i -ésima dessas linhas deve conter dois inteiros x_i, y_i , separados por espaço, indicando que um vínculo deve ser forjado do gato x_i para o gato y_i .

4.3 Limites de execução

Para qualquer caso de teste, seu código deve imprimir a resposta em, no máximo, 3 segundos. Seu programa deve usar menos de 100MB de memória. Estruturas de dados devem ser alocadas sob demanda; ou seja, não faça vetores estáticos gigantescos para grafos com poucos vértices. Todas as avaliações serão feitas automaticamente via VPL. Programas que não aderirem a essas restrições para um teste terão a nota do mesmo zerada.

4.4 Exemplos

Entrada	Saída
4	1
2 3 4 2	4 1

4.4.1 Exemplo 2

Entrada	Saída
4	0
2 3 4 1	

5 Atrasos

O trabalho pode ser entregue com atraso, mas será penalizado de acordo com a seguinte fórmula, onde d é o número de dias atrasados:

$$\Delta(d) = \frac{2^{d-1}}{0.32} \% \quad (1)$$

Por exemplo, com um atraso de quatro dias e uma nota de execução de 70% do total, sua nota final será penalizada em 25%, ficando assim igual a $70 \cdot (1 - \Delta(d)) = 52.5\%$. Note que a penalização é exponencial, e um atraso de 6 dias equivale a uma penalidade de 100%.

6 Errata

- Sem erros por enquanto.